

Components in React

◆ 1. Components in React (Frontend)

This is the **most important place** where the term is used.

A React component is:

- A **piece of UI**
- **Reusable**
- Can manage its own data (state)

✚ Example

```
function Header() {  
  return <h1>My Website</h1>;  
}
```

✚ Types of Components

1. **Functional Components (Modern)**
2. **Class Components (Older)**

◆ Real MERN Example

Imagine a blog app:

- Navbar → component
- PostList → component
- PostItem → component
- Footer → component

👉 You combine these to build the full UI.

◆ 2. Component-Based Architecture (Frontend Structure)

Your React app is structured like:

```
App.js  
├── Navbar.js  
├── Sidebar.js  
├── PostList.js  
│   └── PostItem.js  
└── Footer.js
```

👉 Each file = one **component**

◆ 3. Backend “Components” in MERN

Even though we don’t always call them “components,” the backend is also modular.

Using Node.js and Express:

📌 Backend Structure

/routes
/controllers
/models
/middleware

◆ These act like components:

- **Routes** → define API endpoints
- **Controllers** → handle logic
- **Models** → define database structure (MongoDB)
- **Middleware** → reusable functions (auth, logging)

👉 Each part is reusable → so conceptually they are **components**

◆ 4. Example Flow in MERN

Let’s connect everything:

1. User clicks a button in a **React component**
 2. That component calls an API
 3. API hits an **Express route**
 4. Route calls a **controller**
 5. Controller interacts with **MongoDB**
 6. Data comes back → React component updates UI
-

◆ 5. Why Components Are Important

- ✓ Reusability
- ✓ Clean code
- ✓ Easy maintenance
- ✓ Scalable apps

Without components, MERN apps become messy very quickly.

✔ Simple Summary

In MERN, a **component** is a reusable building block:

- **Frontend:** UI pieces in React
- **Backend:** modular parts like routes, controllers, and models